VECTOR

Synchronization between AUTOSAR Cry and Fls

Version 1.01.00
2022-01-24
Application Note AN-ISC-8-1207

Author Bernhard Wissinger

Restrictions Customer Confidential – Vector decides

Abstract AUTOSAR Crypto and Flash Driver might access a common hardware resource. This application note describes possible synchronization mechanisms.

Table of Contents

1 Introduction 2
1.1 Access Conflict Types 2
1.2 Dependency of Access Conflics to the ECU Use Case 3
2 Use Case Description 3
3 Synchronization Methods of SecOC and Fls 4
3.1 Prevent Execution of SecOC_MainFunction 4
3.2 Prevent Execution of Cry Function 5
3.3 Optimized Execution Prevention 5
3.4 Interrupt Fls Operation 6
4 Synchronization Method for Key Management 7
4.1 Alternative as KeyM Does Not Have Retry Function 7
5 Contacts 7

1 Introduction

AUTOSAR defines Crypto Driver (Cry) and Flash Driver (Fls) as MCAL modules for independent hardware modules. But as the crypto module also provides key storage functionality, this is often implemented in the microcontroller by using a shared flash unit. This can lead to race conditions, if AUTOSAR Crypto Driver and Flash Driver are used concurrently.

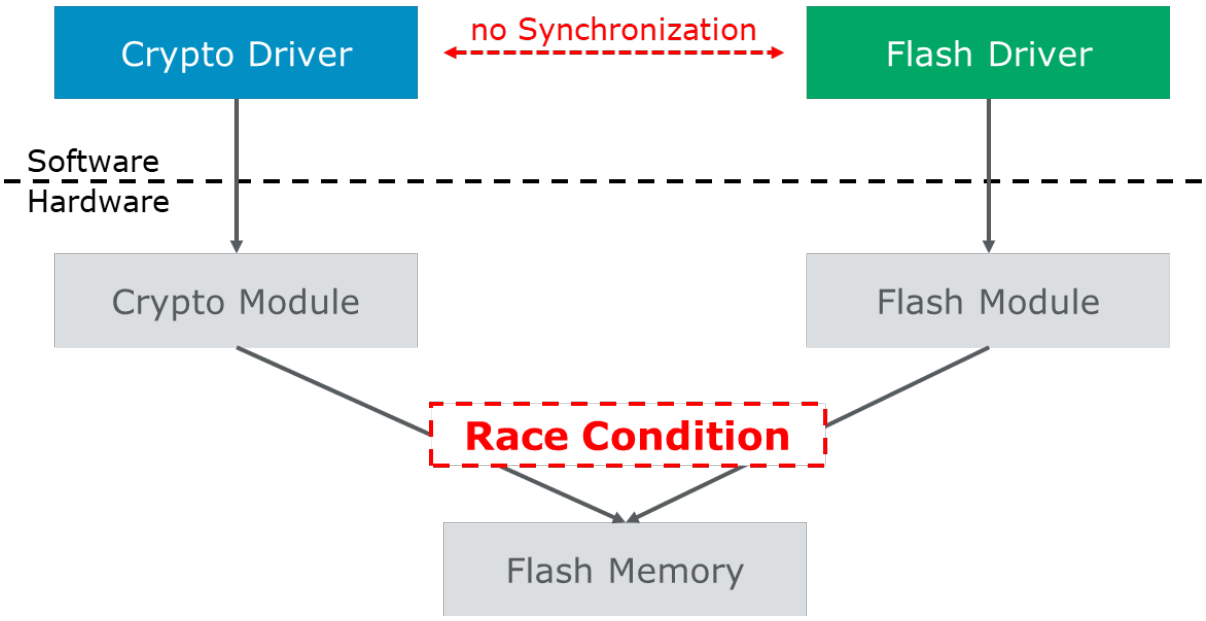


Figure 1 - Race Condition During Flash Memory Access

This application note describes several synchronization methods and gives advice which method should be applied. The synchronization itself must be implemented by the application during the integration of the MICROSAR stack, as the used method depends on the system layout and requirements.

Caution
The occurrence and effect of the race condition heavily depends on the used microcontroller. Please double-check whether your specific hardware is affected and confirm whether the methods described in this application note are applicable to your system.

Caution
The examples in this application note are not thoroughly tested. The user must verify the functionality for the intended use case. Vector´s liability shall be expressly excluded to the extent admissible by law or statute.

1.1 Access Conflict Types

Table 1 lists possible access conflict types. In this application note, it is assumed that each of these combinations will lead to a race condition and must be prevented. E.g. even if the Crypto driver performs only a read access to the key (like MacVerify), a parallel read access of the flash driver to the hardware is not allowed.

If the used microcontroller is less restrictive, a less restrictive synchronization scheme might be applied. Such optimized synchronization scheme is not covered by this application note.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

Crypto Driver	Flash Driver	Conflict
Use Key (e.g. MacVerify, MacGenerate)	Read Flash Memory	Read / Read
Write Key (e.g. KeyElementSet)	Read Flash Memory	Write / Read
Use Key (e.g. MacVerify, MacGenerate)	Write Flash Memory	Read / Write
Write Key (e.g. KeyElementSet)	Write Flash Memory	Write / Write

Table 1 - Access Conflict Types

1.2 Dependency of Access Conflics to the ECU Use Case

The possibility of an access conflict heavily depends on the ECU use case. As an access conflict can only happen if the Flash module and Crypto module are used at the same time, a detailed analysis of this use case is needed.

> E.g. if the Crypto module is used only during ECU run state for secure communication, whereas Flash module access is limited to ECU startup and shutdown, no access conflict might occur.
> The access conflict might be limited to system startup, in case the Crypto module can cache the keys in secure RAM and therefore has no need to access data flash during later usage.

The potential access conflicts need to be analyzed by the user. This application note gives an example how to solve such a conflict for secure communication and data flash access.



Caution
The occurrence and effect of access conflicts heavily depend on the ECU use case. Please analyze the specific Crypto and Flash use cases in your ECU to confirm potential conflicts and to judge for the appropriate resolution method.

2 Use Case Description

The synchronization methods are discussed based on three different users of Crypto and Flash Driver. Please adapt the described concepts in case of a different use case shall be implemented.



Caution
The described behavior of the Fls module is implementation-specific. Please confirm this behavior for the used Fls module.

User	Functionality	Call context of hardware access
Nonvolatile Memory	Fls: Read and Write	Fls_MainFunction
SecOC	Cry: MAC Processing	SecOC_MainFunction
Key Management	Cry: Key Update	KM_MainFunction

Table 2 - Users of Crypto Driver and Flash Driver

The Fls is used asynchronously: the flash operation is started in `Fls_MainFunction`, but will be finished later.

The Cry is used synchronously: the crypto hardware is idle after the execution of `SecOC_MainFunction` and `KM_MainFunction` is completed.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

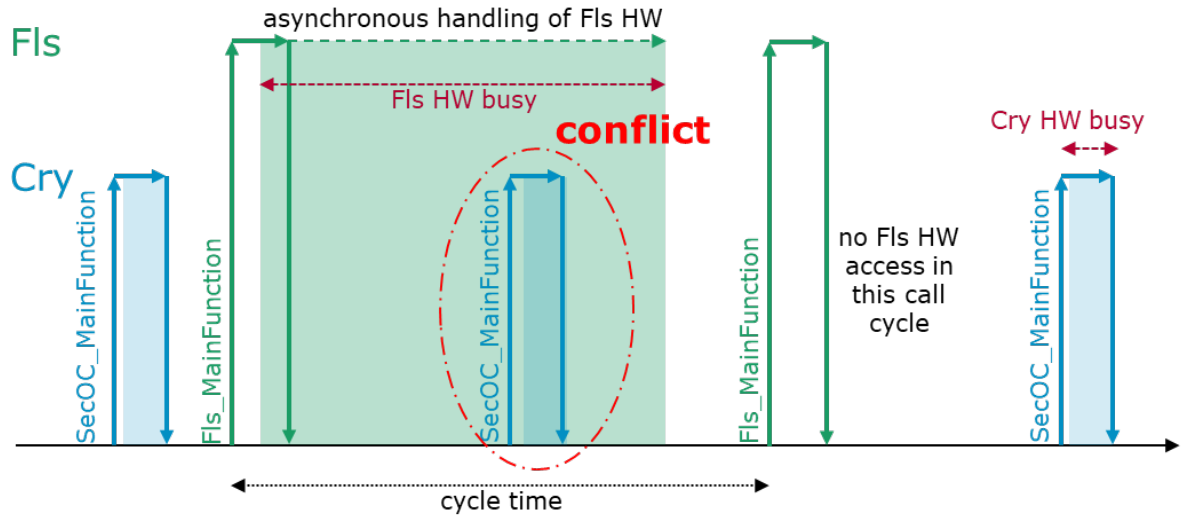


Figure 2 - Conflict Between Fls and Cry

3 Synchronization Methods of SecOC and Fls

This chapter describes synchronization methods between Fls and Cry for the SecOC use case.

Caution
It is assumed that `Fls_MainFunction` cannot interrupt `SecOC_MainFunction`. E.g. `Fls_MainFunction` is mapped to the same or a lower priority OS task as `SecOC_MainFunction`.

Table 3 gives an overview and comparison of the different methods which are described in this chapter.

Method	Characteristics	AUTOSAR Extensions
3.1	SecOC messages will not be sent during Fls operation. Especially Fls erase might take a long time.	None
3.2	Same as 3.1	Cry extension “Read Start” interface
3.3	Less impact on SecOC messages for “short” Fls operations	Fls extension “GetHWStatus”
3.4	Difficult if Suspend/Resume needs a long time	Fls extension “Suspend/Resume”

Table 3 – Comparison Matrix

3.1 Prevent Execution of SecOC_MainFunction

The current state of Fls can be polled by the function `Fls_GetStatus`. Therefore `SecOC_MainFunction` should not be called in case `Fls_GetStatus` returns “busy”.

Caution
It is assumed that `Fls_GetStatus` can be called reentrant. Please confirm this behavior for the used Fls module.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

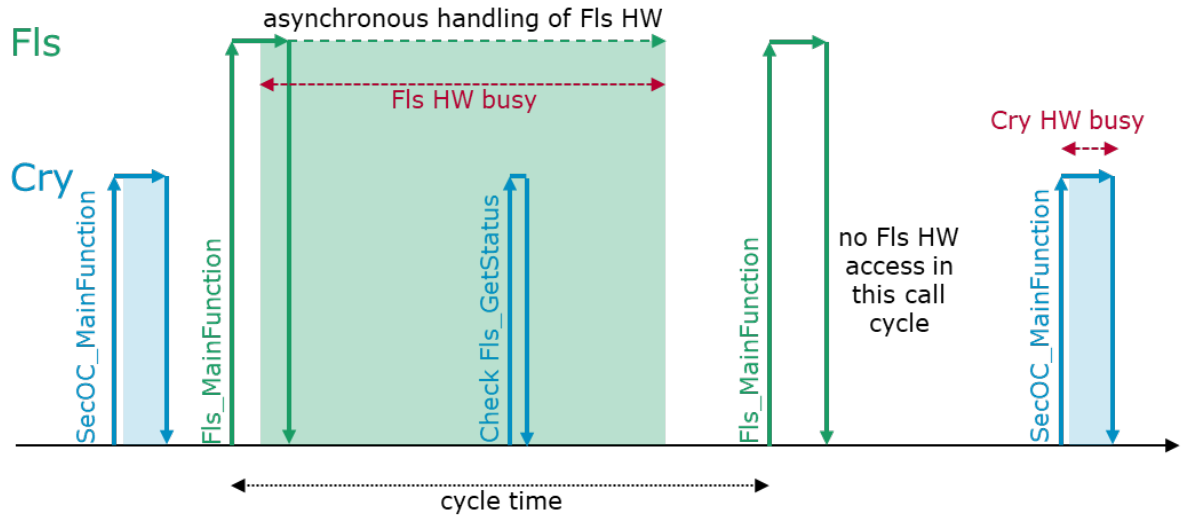


Figure 3 - Prevent Execution of SecOC_MainFunction

3.2 Prevent Execution of Cry Function

This approach is similar to chapter 3.1. But instead of preventing the execution of `SecOC_MainFunction`, an API like `Cry_XXX_DataFlashReadStart_Callout` would be used. With the API `Cry_XXX_DataFlashReadStart_Callout`, the read permission is checked by the Cry driver: this shall be denied in case the Fls is busy.

The `SecOC_MainFunction` will retry with the next cyclic call, if the parameter `SecOCAuthenticationBuildAttempts` in SecOC is set accordingly.

Note
The API `Cry_XXX_DataFlashReadStart_Callout` is not defined by AUTOSAR and might not be available for your hardware.

3.3 Optimized Execution Prevention

The API `Fls_GetStatus` returns the status of the Fls driver. Whereas this is implementation-specific, it might be only updated in context of `Fls_MainFunction`. Therefore it would report still “busy”, whereas the operation in the Fls hardware might be already finished.

It is assumed an API like `Fls_GetHWStatus` is available, which reads the status of the Fls hardware itself. An optimized version of methods in chapter 3.1 and chapter 3.2 can be applied in this case:

- In task scheduling, `SecOC_MainFunction` is executed immediately before `Fls_MainFunction`. Therefore the Fls operation from the previous `Fls_MainFunction` might be already completed.
- The API `Fls_GetHWStatus` is used instead of `Fls_GetStatus`

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

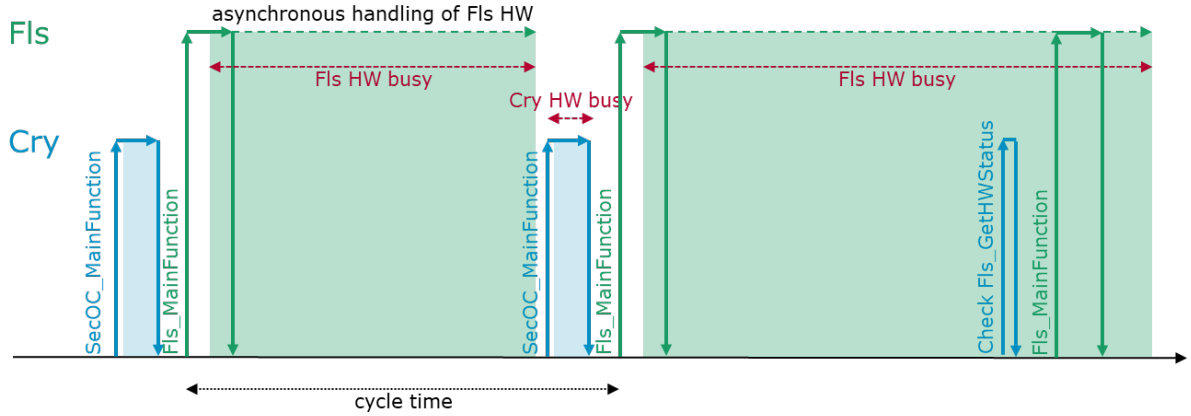


Figure 4 - Optimized Execution Prevention

With this approach, Cry handling is only skipped if the Fls needs a long processing time.

3.4 Interrupt Fls Operation

Some Fls drivers have a possibility to suspend and resume the current Fls operation. Such functionality is used for this approach:

- In task scheduling, `SecOC_MainFunction` is executed immediately before `Fls_MainFunction`. Therefore the Fls operation from the previous `Fls_MainFunction` might be already completed.
- In case Fls is busy: Fls operation is suspended before `SecOC_MainFunction` is called
- If Fls operation was suspended: resume Fls operation after `SecOC_MainFunction` is completed

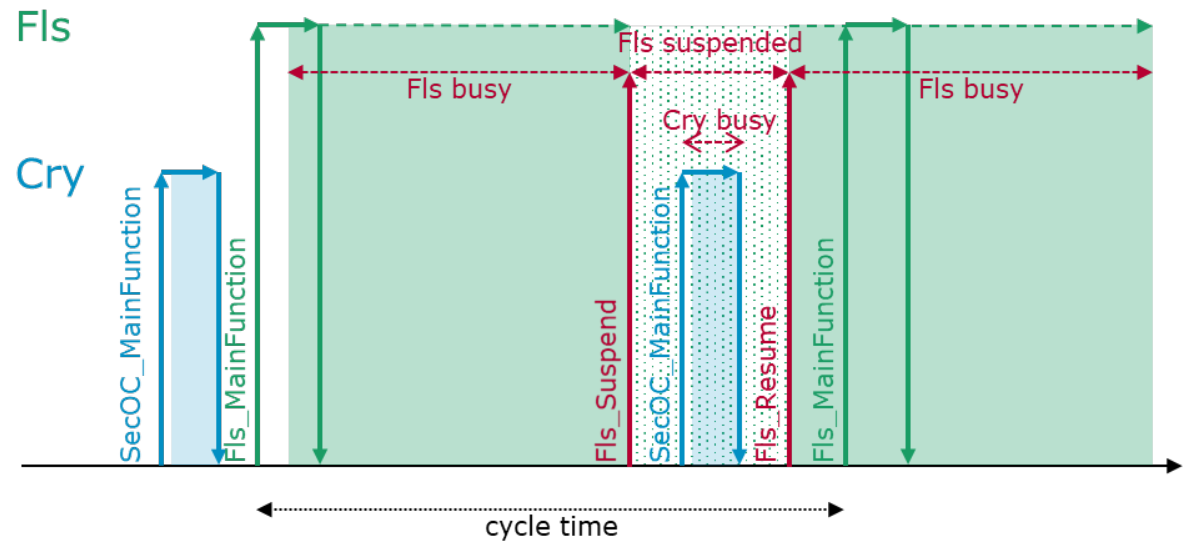


Figure 5 - Interrupt Fls Operation

Caution
The `Fls_Suspend/Fls_Resume` operation itself might need considerable time. This can lead to issues, if a high call frequency of `SecOC_MainFunction` of `Fls_MainFunction` is needed.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0

4 Synchronization Method for Key Management

Synchronization between SecOC and Key Management is not needed, as both utilize the same Cry driver. Therefore the Cry driver should take care of this synchronization.



Caution
It is assumed that Fls_MainFunction cannot interrupt KM_MainFunction. E.g. Fls_MainFunction is mapped to the same or a lower priority OS task as KM_MainFunction.

Synchronization of Key Management and Fls is less time critical than the synchronization between SecOC and Fls. Therefore simpler methods as described in chapter 3.1 and chapter 3.2 can be applied.

If chapter 3.2 is applied, a callback form Cry driver which queries the write permission should be used. Such an API `Cry_XXX_DataFlashWriteStart_Callout` is implemented as an AUTOSAR extension for some Cry drivers developed by Vector.

4.1 Alternative as KeyM Does Not Have Retry Function

However, when applying 3.2, unlike SecOC, KeyM does not have a retry function, so an alternative plan is necessary here. The following methods are available as alternatives.

Cry_XXX_Start_Callout

1. Disable scheduling of Fee/Fls main functions
2. Call main functions of Fee/Fls until not busy anymore
3. Return TRUE

Cry_XXX_End_Callout

1. Enable scheduling of Fee/Fls main functions

The callouts of the Cry driver allow to perform a synchronous write(read) by waiting inside the callout until the flash access is done.

Keep in mind that disabling the Fls/Fee scheduling is always mandatory to ensure that Cry is the only entity using the flash during key writing.

Waiting for an ongoing flash operation to end is optional, but then make sure with scheduling, that it is ensured that no ongoing flash operation happens when trying to write a key.

5 Contacts

For a full list with all Vector locations and addresses worldwide, please visit http://vector.com/contact/.

Copyright © 2022 - Vector Informatik GmbH

Contact Information: www.vector.com or +49-711-80 670-0